NumPy – consulta rápida 1 · Arrays

01 CRIAR ARRAYS

cap. 22 · §2–3

`a = np.array([1, 2, 3, 4, 5])`	A partir de uma lista (1D).
`m = np.array([[1, 2], [3, 4], [5, 6]])`	Matriz 3×2: uma lista por linha.
`np.zeros((2, 3))` `np.ones(3)`	Zeros · uns; a forma num tuplo.
`np.full((2, 2), 7)` `np.eye(3)`	Tudo a 7 · matriz identidade.
`np.arange(0, 10, 2)`	`[0 2 4 6 8]`: o fim não entra.
`np.linspace(0, 1, 5)`	5 valores de 0 a 1, com o fim.

02 INSPECIONAR E TIPOS

cap. 22 · §4–5

`m.shape` `m.ndim` `m.size`	`(3, 2)` · 2 eixos · 6 elementos.
`m.dtype`	Tipo único de todos os elementos.
`np.array([1, 2.5]).dtype`	`float64`: o tipo mais geral.
`a.astype(float)`	Converter; devolve um array novo.

03 SELECIONAR

cap. 22 · §6

`a[0]` `a[-1]` `a[1:4]`	Primeiro · último · posições 1 a 3.
`m[0, 1]`	Linha 0, coluna 1 → 2.
`m[1]` `m[:, 1]`	Uma linha · uma coluna (ficam 1D).
`m[:, 1:2]`	Coluna em 2D: forma `(3, 1)`.

04 MÁSCARAS

cap. 22 · §7

`a[a > 2]`	Só os que cumprem a condição.
`a[(a > 1) & (a < 5)]`	`&` e · `|` ou · `~` não, com parênteses.
`a[a < 0] = 0`	Alterar só os selecionados.
`np.where(a > 2, a, 0)`	Valor se sim · valor se não.

05 CÓPIAS E VISTAS

cap. 22 · §8

`v = a[1:3]`	Vista: alterar `v` altera `a`.
`c = a[1:3].copy()`	Cópia independente.
`np.shares_memory(a, v)`	Partilham memória? → `True`.

06 MUDAR A FORMA

cap. 22 · §9

`a.reshape(2, 3)` `a.reshape(-1, 1)`	Nova forma · em coluna.
`a[:, None]` `a[None, :]`	Novo eixo: coluna · linha.
`m.ravel()` `m.flatten()`	Para 1D: vista · cópia.
`m.T`	Transposta: `(3, 2)` → `(2, 3)`.

07 JUNTAR E DIVIDIR

cap. 22 · §10

`np.concatenate([x, y])`	Juntar num eixo existente.
`np.stack([x, y])`	Empilhar criando um eixo novo.
`np.column_stack([x, y])`	Vetores lado a lado, como colunas.
`np.split(a, 3)`	Partir em 3 partes iguais.

08 ALEATORIEDADE

cap. 22 · §14

`rng = np.random.default_rng(42)`	Semente fixa: repetível.
`rng.integers(1, 7, size=10)`	10 inteiros de 1 a 6 (o 7 não entra).
`rng.normal(0, 1, size=(3, 2))`	Normal: média 0, desvio 1.

09 GUARDAR E LER

cap. 22 · §20

`np.save("a.npy", a)` `np.load("a.npy")`	Guarda a forma e o tipo.
`np.savetxt("m.csv", m, delimiter=",")`	Texto. Para ler: `np.loadtxt`.

NumPy – consulta rápida

2 · Cálculo e armadilhas

Capítulo 22
`a`: vetor · `m`: matriz 3×2

10 OPERAÇÕES VETORIZADAS

cap. 22 · §19

`a + 1` `a * 2` `a ** 2`	Aplica-se a cada elemento.
`x + y` `x * y`	Elemento a elemento.
`np.sqrt(a)` `np.exp(a)` `np.log(a)`	Funções sobre o array todo.
`np.round(a, 2)` `np.abs(a)`	Arredondar · valor absoluto.

11 BROADCASTING

cap. 22 · §11

`m - m.mean(axis=0)`	Centrar cada coluna.
`(3, 2) + (2,)`	Compatível: alinha pela direita.
`(3, 2) + (3,)`	Erro: 2 e 3 não coincidem.
`m + v.reshape(-1, 1)`	Forma `(3, 1)`: um valor por linha.

12 EIXOS E AGREGAÇÕES

cap. 22 · §12

`a.sum()` `a.mean()` `a.std()` `a.min()` `a.max()`	Resumos de todo o array.
`m.sum(axis=0)`	Reduz as linhas: um por coluna.
`m.sum(axis=1)`	Reduz as colunas: um por linha.
`m.mean(axis=1, keepdims=True)`	Mantém o eixo: forma `(3, 1)`.
`a.argmax()` `a.argmin()`	Posição do maior · do menor.
`np.cumsum(a)`	Soma acumulada.

13 ORDENAR E CONTAR

cap. 22 · §13

`np.sort(a)` `a.argsort()`	Ordenado · posições da ordem.
`np.unique(a, return_counts=True)`	Valores distintos e contagens.

14 VALORES ESPECIAIS

cap. 22 · §15

`np.nan` `np.inf`	Ausente · infinito.
`np.isnan(a).sum()`	Contar os ausentes.
`np.nanmean(a)` `np.nansum(a)`	Calcular ignorando `NaN`.
`np.allclose(a, b)` `np.isclose(x, y)`	Iguais com tolerância.

15 ÁLGEBRA LINEAR

cap. 22 · §16

`A @ B`	Produto matricial.
`np.dot(u, v)`	Produto interno de vetores.
`np.linalg.solve(A, b)`	Resolver `A · x = b`.
`np.linalg.norm(v)`	Comprimento do vetor.

16 PADRONIZAR

cap. 22 · §17

`(x - x.mean(axis=0)) / x.std(axis=0)`	Média 0 e desvio 1 por coluna.
`(x - x.min(axis=0)) /` `(x.max(axis=0) - x.min(axis=0))`	Min–max: valores de 0 a 1.

! ARMADILHAS FREQUENTES

✗ errado · ✓ certo

Lista ou array. Numa lista, `*` repete os elementos.

```
✗ [1, 2] * 2          # [1, 2, 1, 2]
✓ np.array([1, 2]) * 2   # [2 4]
```

Multiplicar matrizes. `*` multiplica elemento a elemento.

```
✗ A * B
✓ A @ B
```

Combinar condições. `and`/`or` não funcionam com arrays.

```
✗ a[a > 1 and a < 5]
✓ a[(a > 1) & (a < 5)]
```

Converter para inteiro. `astype(int)` corta as casas decimais.

```
✗ np.array([2.7]).astype(int)        # 2
✓ np.round(np.array([2.7])).astype(int)   # 3
```

`axis=0` percorre as linhas e dá um resultado por coluna; `axis=1` percorre as colunas e dá um por linha.